

Circuitos Sequenciais

Implementação em VHDL

Prof. Roberto de Matos

roberto.matos@ifsc.edu.br



**INSTITUTO
FEDERAL**
Santa Catarina

Câmpus
São José

- Inferir elementos de memória em VHDL
- Entender exemplos de implementação em VHDL usando o método síncrono
- Comparar estilos de codificação
- Diretrizes de síntese



Objetivo

- Inferir elementos de memória em VHDL
- Entender exemplos de implementação em VHDL usando o método síncrono
- Comparar estilos de codificação
- Diretrizes de síntese

Leitura Recomendada:

Capítulo 8 do livro: *RTL Hardware Design Using VHDL*



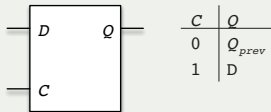
Relembrando ...

- Sequential vs Combinacional:
 - Sequential tem estado interno.
 - Saída é função das entradas e do estado.
- Elementos básicos de memória
 - Latch D
 - Flip-Flop D
- Podem ser síncrono ou assíncrono.

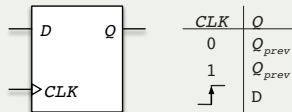


Latch D e Flip-Flop D (DFF)

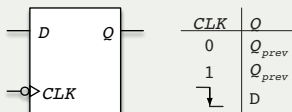
■ Latch D:



■ DFF sensível à borda de subida:



■ DFF sensível à borda de descida:



■ DFF com reset assíncrono e enable:

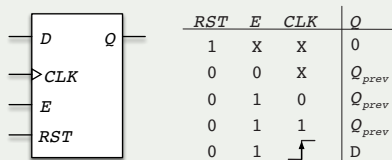
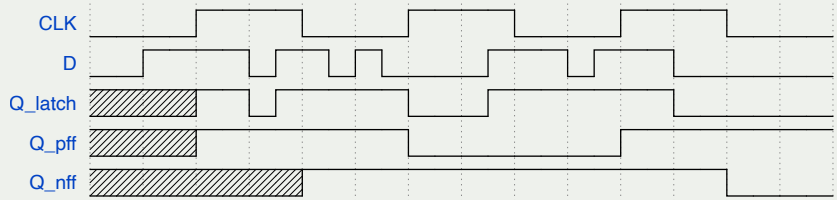
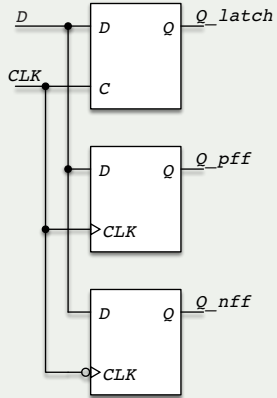


Diagrama de tempo simplificado



- Um dos aspectos de projeto mais difíceis de um circuito sequencial:
 - Como satisfazer as restrições (*constraints*) de tempo (ex.: T_{setup} e T_{hold}).

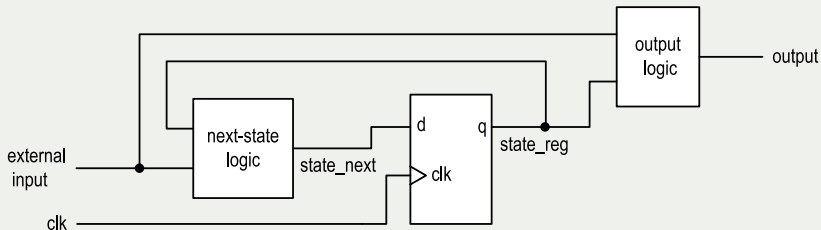


- Um dos aspectos de projeto mais difíceis de um circuito sequencial:
 - Como satisfazer as restrições (*constraints*) de tempo (ex.: T_{setup} e T_{hold}).
- A grande ideia: **MÉTODO SÍNCRONO**
 - Agrupe todos os DFFs em um único *clock*.
 - Só precisa lidar com a restrição de tempo de um elemento de memória.



Método Síncrono

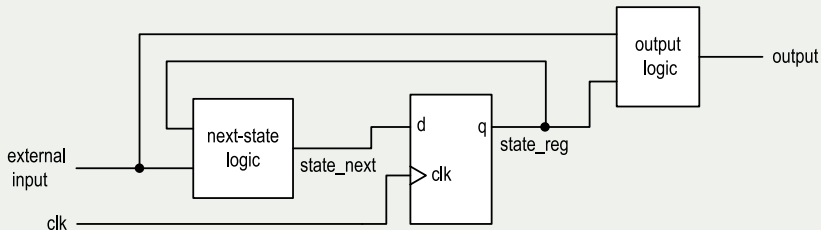
- Diagrama conceitual:
 - Registrador de estado (elemento de memória)
 - Lógica do próximo estado (circuito combinacional)
 - Lógica de saída (circuito combinacional)



Método Síncrono

■ Diagrama conceitual:

- Registrador de estado (elemento de memória)
- Lógica do próximo estado (circuito combinacional)
- Lógica de saída (circuito combinacional)



Questão:

Como descrever um elemento de memória?

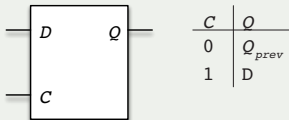


Inferência de elementos de memória básicos

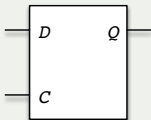
- O código VHDL deve ser claro para que as células pré-projetadas possam ser inferidas.
- Código VHDL
 - *Latch D*
 - *Flip Flop D* sensível à borda de subida
 - *Flip Flop D* sensível à borda de descida
 - *Flip Flop D* com reset assíncrono.



- Sem cláusula *else*
- O *Latch D* será inferido automaticamente



- Sem cláusula *else*
- O Latch D será inferido automaticamente

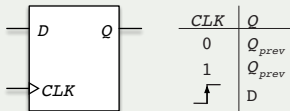


<i>C</i>	<i>Q</i>
0	<i>Q_{prev}</i>
1	<i>D</i>

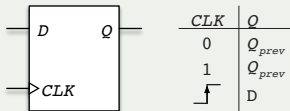
```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  entity dlatch is
4      port(
5          c: in std_logic;
6          d: in std_logic;
7          q: out std_logic
8      );
9  end dlatch;
10
11 architecture arch of dlatch is
12 begin
13     process(c,d)
14     begin
15         if (c='1') then
16             q <= d;
17         end if;
18     end process;
19 end arch;
```



- Sem cláusula e/se
- Perceba a lista de sensibilidade



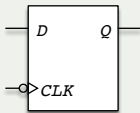
- Sem cláusula *else*
- Perceba a lista de sensibilidade



```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 entity dff is
4     port(
5         clk: in std_logic;
6         d: in std_logic;
7         q: out std_logic
8     );
9 end dff;
10
11 architecture arch of dff is
12 begin
13     process(clk)
14     begin
15         if (clk'event and clk='1') then
16             q <= d;
17         end if;
18     end process;
19 end arch;
```



■ Mudança de apenas uma linha



CLK	Q
0	Q_{prev}
1	Q_{prev}
	D

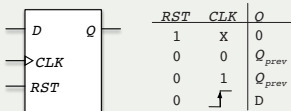
15

```
if (clk'event and clk='0') then
```



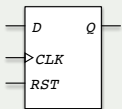
Flip Flop D com reset assíncrono

- Sem cláusula *else*
- Perceba a lista de sensibilidade



Flip Flop D com reset assíncrono

- Sem cláusula *else*
- Perceba a lista de sensibilidade

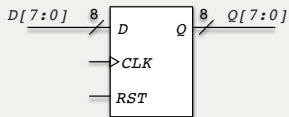


<i>RST</i>	<i>CLK</i>	<i>Q</i>
1	X	0
0	0	<i>Q_{prev}</i>
0	1	<i>Q_{prev}</i>
0		D

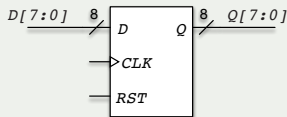
```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 entity dffr is
4     port(
5         clk: in std_logic;
6         reset: in std_logic;
7         d: in std_logic;
8         q: out std_logic
9     );
10 end dffr;
11
12 architecture arch of dffr is
13 begin
14     process(clk,reset)
15     begin
16         if (reset='1') then
17             q <='0';
18         elsif (clk'event and clk='1') then
19             q <= d;
20         end if;
21     end process;
22 end arch;
```



- Múltiplos *Flip Flops* D com o mesmo *clock* e *reset*



■ Múltiplos *Flip Flops* D com o mesmo clock e reset



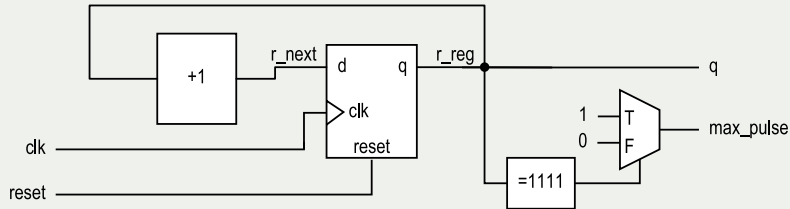
```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 entity reg8 is
4     port(
5         clk: in std_logic;
6         reset: in std_logic;
7         d: in std_logic_vector(7 downto 0);
8         q: out std_logic_vector(7 downto 0)
9     );
10 end reg8;
11
12 architecture arch of reg8 is
13 begin
14     process(clk,reset)
15     begin
16         if (reset='1') then
17             q <=(others=>'0');
18         elsif (clk'event and clk='1') then
19             q <= d;
20         end if;
21     end process;
22 end arch;
```



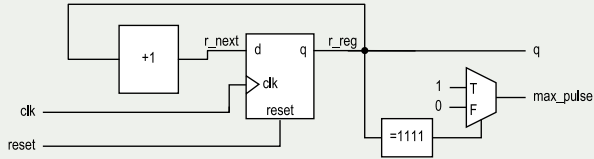
Exemplos

Exemplo 1: Contador binário livre com pulso de máximo

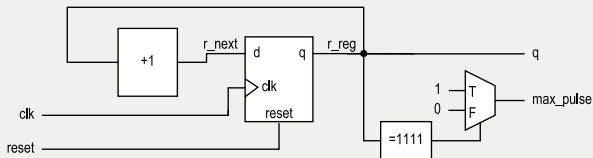
- Contagem em sequência binária
- Com uma saída `max_pulse`: pulso quando o contador está no estado “11 ... 11”
- Exemplo com 4 bits:



Exemplo 1: Contador binário livre com pulso de máximo - VHDL



Exemplo 1: Contador binário livre com pulso de máximo - VHDL



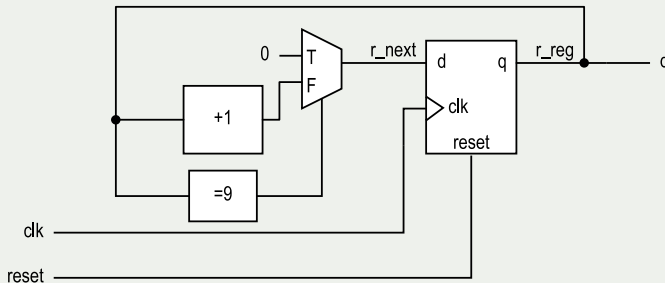
```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4 entity binary_counter4_pulse is
5     port(
6         clk, reset: in std_logic;
7         max_pulse: out std_logic;
8         q: out std_logic_vector(3 downto 0)
9     );
10 end binary_counter4_pulse;
```

```
11 architecture two_seg_arch of binary_counter4_pulse is
12     signal r_reg: unsigned(3 downto 0);
13     signal r_next: unsigned(3 downto 0);
14 begin
15     -- register
16     process(clk,reset)
17     begin
18         if (reset='1') then
19             r_reg <= (others=>'0');
20         elsif (clk'event and clk='1') then
21             r_reg <= r_next;
22         end if;
23     end process;
24     -- next-state logic
25     r_next <= r_reg + 1;
26     -- output logic
27     q <= std_logic_vector(r_reg);
28     max_pulse <= '1' when r_reg="1111" else
29         '0';
30 end two_seg_arch;
```

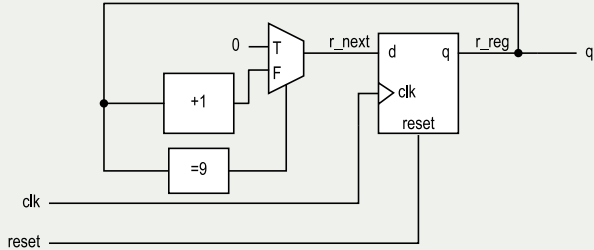


Exemplo 2: Contador decimal (*mod-10*)

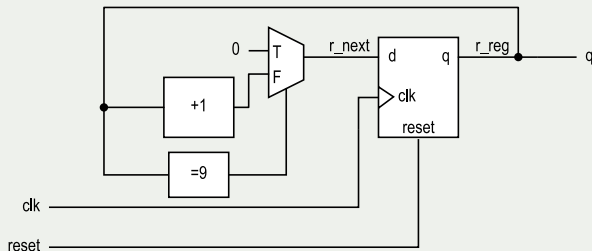
- Contador de 0 a 9.
- Como *mod-10* não é potência de 2, é necessário reiniciar a contagem após o 9.
- Necessário pelo menos 4 bits ($\lceil \log_2 10 \rceil$).



Exemplo 2: Contador decimal (*mod-10*) - VHDL



Exemplo 2: Contador decimal (*mod-10*) - VHDL

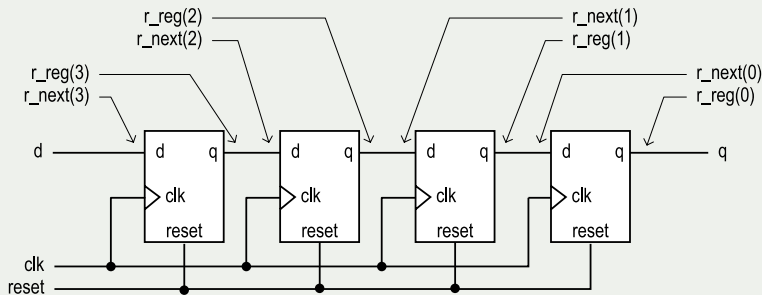


```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4 entity mod10_counter is
5     port(
6         clk, reset: in std_logic;
7         q: out std_logic_vector(3 downto 0)
8     );
9 end mod10_counter;
10
11 architecture two_seg_arch of mod10_counter is
12     constant TEN: integer := 10;
13     signal r_reg: unsigned(3 downto 0);
14     signal r_next: unsigned(3 downto 0);
15 begin
16     -- register
17     process(clk,reset)
18     begin
19         if (reset='1') then
20             r_reg <= (others=>'0');
21         elsif (clk'event and clk='1') then
22             r_reg <= r_next;
23         end if;
24     end process;
25     -- next-state logic
26     r_next <= (others=>'0') when r_reg=(TEN-1) else
27         r_reg + 1;
28     -- output logic
29     q <= std_logic_vector(r_reg);
30 end two_seg_arch;
```

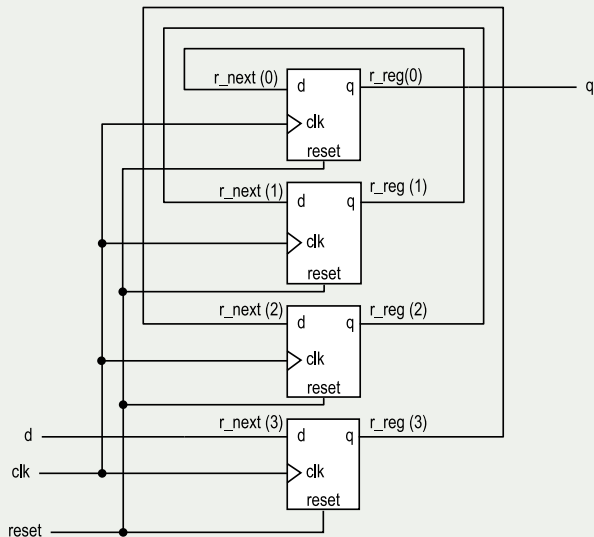


Exemplo 3: Registrador de deslocamento livre

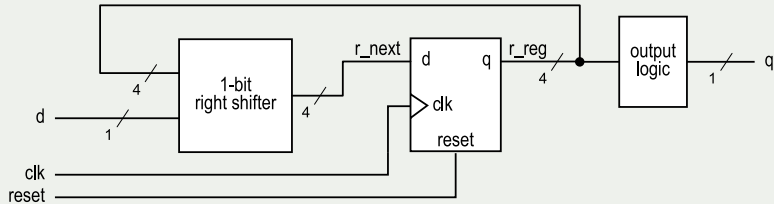
- Um registrador de deslocamento livre executa a operação de deslocamento continuamente.
- Não tem outros sinais de controle.



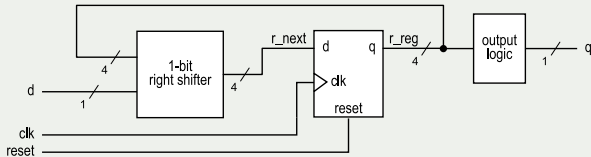
Exemplo 3: Registrador de deslocamento livre - forma vertical



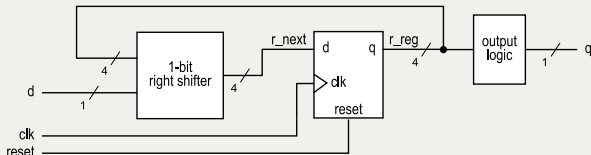
Exemplo 3: Registrador de deslocamento livre - forma básica



Exemplo 3: Registrador de deslocamento livre - VHDL



Exemplo 3: Registrador de deslocamento livre - VHDL



```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 entity shift_right_register is
4     port(
5         clk, reset: in std_logic;
6         d: in std_logic;
7         q: out std_logic
8     );
9 end shift_right_register;
10
11 architecture two_seg_arch of shift_right_register is
12     signal r_reg: std_logic_vector(3 downto 0);
13     signal r_next: std_logic_vector(3 downto 0);
14 begin
15     -- register
16     process(clk,reset)
17     begin
18         if (reset='1') then
19             r_reg <= (others=>'0');
20         elsif (clk'event and clk='1') then
21             r_reg <= r_next;
22         end if;
23     end process;
24     -- next-state logic (shift right 1 bit)
25     r_next <= d & r_reg(3 downto 1);
26     -- output
27     q <= r_reg(0);
28 end two_seg_arch;
```

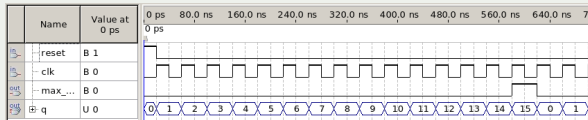


Estilo de codificação

DOIS segmentos vs. UM segmento

- Exemplo: Contador binário livre com pulso de máximo
- Arquivo list_08_12_20_21_bin_counter.vhd
- Original de DOIS segmentos:

```
4 library ieee;
5 use ieee.std_logic_1164.all;
6 use ieee.numeric_std.all;
7 entity binary_counter4_pulse is
8     port(
9         clk, reset: in std_logic;
10         max_pulse: out std_logic;
11         q: out std_logic_vector(3 downto 0)
12     );
13 end binary_counter4_pulse;
```



```
15 architecture two_seg_arch of binary_counter4_pulse is
16     signal r_reg: unsigned(3 downto 0);
17     signal r_next: unsigned(3 downto 0);
18 begin
19     -- register
20     process(clk,reset)
21     begin
22         if (reset='1') then
23             r_reg <= (others=>'0');
24         elsif (clk'event and clk='1') then
25             r_reg <= r_next;
26         end if;
27     end process;
28     -- next-state logic
29     r_next <= r_reg + 1;
30     -- output logic
31     q <= std_logic_vector(r_reg);
32     max_pulse <= '1' when r_reg="1111" else
33         '0';
34 end two_seg_arch;
```



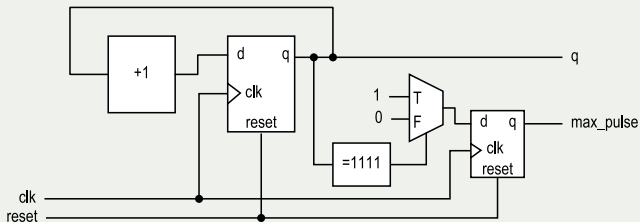
■ Código de UM segmento (**incorreto**):

```
37 -----
38 -- Listing 8.20
39 -----
40 architecture not_work_one_seg_glitch_arch
41     of binary_counter4_pulse is
42     signal r_reg: unsigned(3 downto 0);
43 begin
44     process(clk,reset)
45     begin
46         if (reset='1') then
47             r_reg <= (others=>'0');
48         elsif (clk'event and clk='1') then
49             r_reg <= r_reg + 1;
50             if r_reg="1111" then
51                 max_pulse <= '1';
52             else
53                 max_pulse <= '0';
54             end if;
55         end if;
56     end process;
57     q <= std_logic_vector(r_reg);
58 end not_work_one_seg_glitch_arch;
```



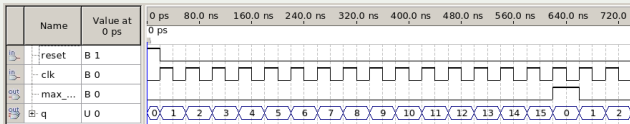
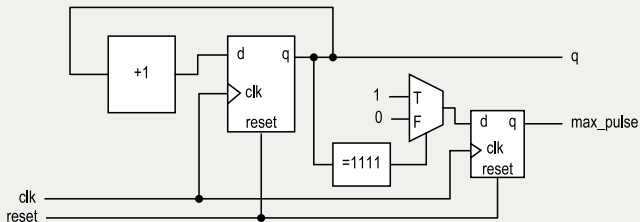
■ Código de UM segmento (**incorreto**):

```
37 -----
38 -- Listing 8.20
39 -----
40 architecture not_work_one_seg_glitch_arch
41     of binary_counter4_pulse is
42     signal r_reg: unsigned(3 downto 0);
43 begin
44     process(clk,reset)
45     begin
46         if (reset='1') then
47             r_reg <= (others=>'0');
48         elsif (clk'event and clk='1') then
49             r_reg <= r_reg + 1;
50             if r_reg="1111" then
51                 max_pulse <= '1';
52             else
53                 max_pulse <= '0';
54             end if;
55         end if;
56     end process;
57     q <= std_logic_vector(r_reg);
58 end not_work_one_seg_glitch_arch;
```



■ Código de UM segmento (**incorreto**):

```
37 -----
38 -- Listing 8.20
39 -----
40 architecture not_work_one_seg_glitch_arch
41     of binary_counter4_pulse is
42     signal r_reg: unsigned(3 downto 0);
43 begin
44     process(clk,reset)
45     begin
46         if (reset='1') then
47             r_reg <= (others=>'0');
48         elsif (clk'event and clk='1') then
49             r_reg <= r_reg + 1;
50             if r_reg="1111" then
51                 max_pulse <= '1';
52             else
53                 max_pulse <= '0';
54             end if;
55         end if;
56     end process;
57     q <= std_logic_vector(r_reg);
58 end not_work_one_seg_glitch_arch;
```



■ Código de UM segmento (**correto**):

```
61 -----
62 -- Listing 8.21
63 -----
64 architecture work_one_seg_glitch_arch
65     of binary_counter4_pulse is
66     signal r_reg: unsigned(3 downto 0);
67 begin
68     process(clk,reset)
69     begin
70         if (reset='1') then
71             r_reg <= (others=>'0');
72         elsif (clk'event and clk='1') then
73             r_reg <= r_reg + 1;
74         end if;
75     end process;
76     q <= std_logic_vector(r_reg);
77     max_pulse <= '1' when r_reg="1111" else
78         '0';
79 end work_one_seg_glitch_arch;
```



DOIS segmentos vs. UM segmento

- Código de dois segmentos
 - Separa o segmento de memória do resto do código;
 - Pode ser um pouco complicado (lógica separada);
 - Tem um mapeamento claro para o componente de hardware.



DOIS segmentos vs. UM segmento

■ Código de dois segmentos

- Separa o segmento de memória do resto do código;
- Pode ser um pouco complicado (lógica separada);
- Tem um mapeamento claro para o componente de hardware.

■ Código de um segmento

- Mistura o segmento de memória e lógica de próximo estado/lógica de saída;
- Em alguns casos pode ser mais compacto;
- Mapeamento de *hardware* não é claro;
- Propenso a erros.



DOIS segmentos vs. UM segmento

- Código de dois segmentos
 - Separa o segmento de memória do resto do código;
 - Pode ser um pouco complicado (lógica separada);
 - Tem um mapeamento claro para o componente de hardware.
- Código de um segmento
 - Mistura o segmento de memória e lógica de próximo estado/lógica de saída;
 - Em alguns casos pode ser mais compacto;
 - Mapeamento de *hardware* não é claro;
 - Propenso a erros.

O código de dois segmentos é preferido .



Conclusão

- Seguir estritamente o método de projeto síncrono; ou seja, todos os registros em um sistema devem ser sincronizados por um sinal de *clock* global comum.
- Isole os componentes da memória da descrição VHDL e codifique-os em um segmento separado. O estilo de codificação de UM segmento não é aconselhável.
- Os componentes da memória devem ser codificados claramente para que uma célula predefinida possa ser inferida da biblioteca do dispositivo.
- Evite sintetizar um componente de memória de lógica combinacional.



Fim!